08

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

싱글 스레드의 경쟁 상태

제1부 기반: 백엔드와 Node.js 런타임 · 2장 이벤트 루프와 비동기 I/O

스레드가 하나인 Node에서 쿠폰이 두 번 쓰이는 이유와 막는 도구



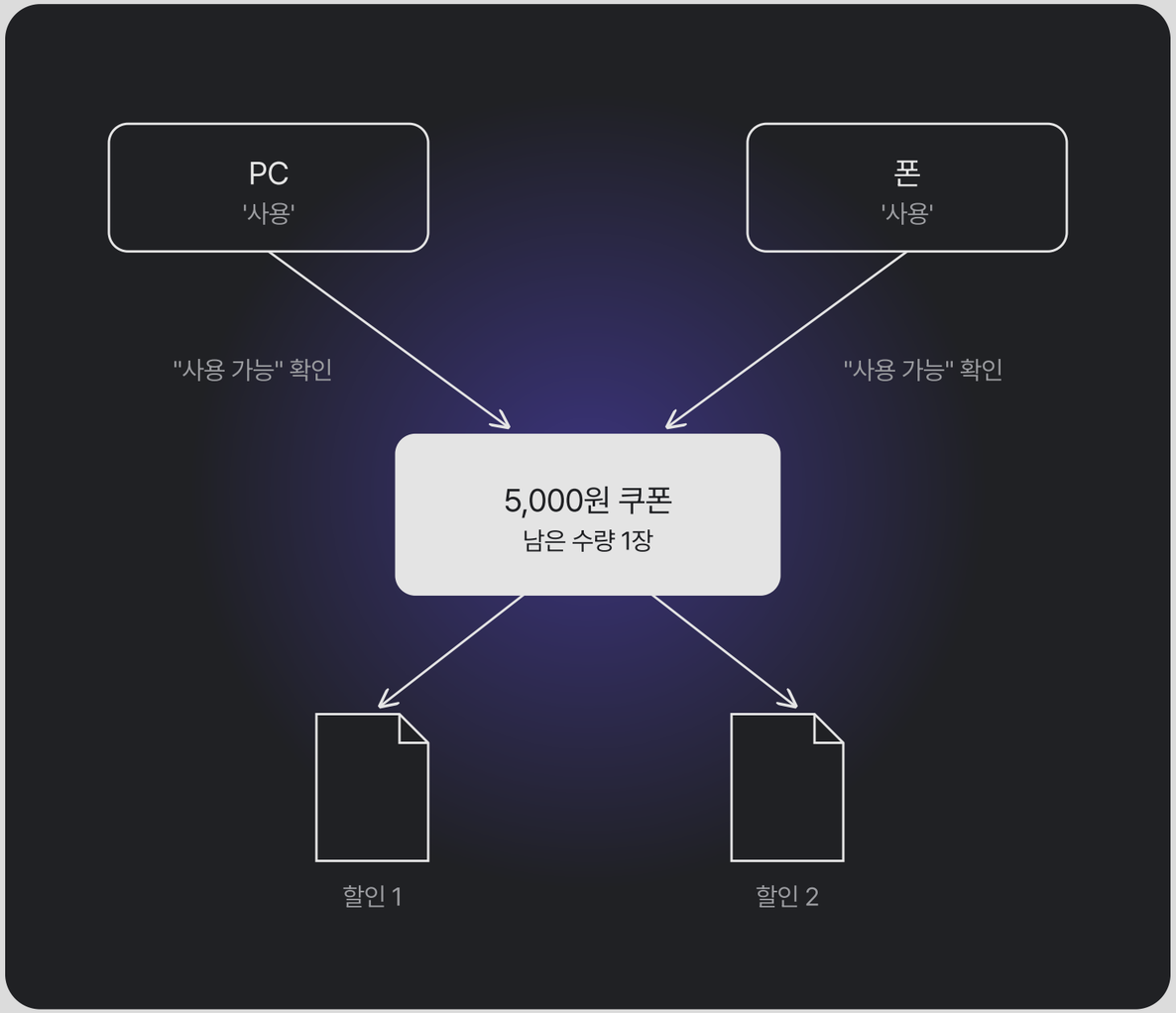
© 한상훈 NestJS 백엔드 마스터

쿠폰 한 장, 버튼 두 개

→ 계정에 남은 5,000원 할인 쿠폰: 1장

→ PC와 폰에서 거의 동시에 '사용'

→ 두 요청 모두 "사용 가능" 확인 → 두 번 할인



© 한상훈 NestJS 백엔드 마스터

잘못된 생각: 싱글 스레드라 안전

→ 앞의 두 줄은 사실, 마지막 줄은 틀림

→ 예: 선착순 1,000명 쿠폰 → 남은 수량 읽기와 차감 사이의 틈 → 1,000장 넘게 발급

1

"Node는 자바스크립트 스레드가 하나"

2

"두 코드가 동시에 같은 변수를 고칠 수 없음"

3

"그러니 경쟁 상태도 없음"

© 한상훈 NestJS 백엔드 마스터

쿠폰 사용 API

coupon-server.mjs

```
// DB 흉내: 읽기와 쓰기에 각각 10ms
const db = {
  async find(code) { await sleep(10); return { ...table.get(code) }; },
  async markUsed(code) { await sleep(10); table.get(code).used = true; },
};

if (url.pathname === '/coupons/use') {
  const code = url.searchParams.get('code');
  const coupon = await db.find(code);          // 1. 읽기
  if (coupon.used) return res.end('이미 사용한 쿠폰\n');
  await db.markUsed(code);                     // 2. 쓰기
  return res.end('5,000원 할인 적용\n');
}
```

© 한상훈 NestJS 백엔드 마스터

동시에 두 번

터미널

```
$ node fire.mjs "/coupons/use?code=WELCOME-5000" 2
/coupons/use?code=WELCOME-5000 { '5,000원 할인 적용': 2 }
```

→ 서버 한 대, Node 프로세스 하나

→ 쿠폰 한 장에 할인 두 번

데이터 블록: 2026-10-26 강사 PC, Node v24.11.0 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

남은 좌석 1,000석, 100건 판매

```
const left = await seatDb.read();              // 읽기
await seatDb.write(left - 1);                  // 쓰기
```

→ 100장 팔림, 숫자는 1만 감소

○ ○ ○ 터미널

```
$ node fire.mjs "/seats/sell" 100
/seats/sell { ok: 100 }
$ curl -s localhost:4008/seats
999
```

© 한상훈 NestJS 백엔드 마스터

로컬 테스트가 통과하는 이유

→ 테스트: 쿠폰 사용 → 결과 확인 → 다시 사용 → "이미 사용한 쿠폰"

→ 요청이 하나씩 → 읽기와 쓰기 사이에 끼어드는 요청 없음

→ 운영: 같은 쿠폰 요청이 10ms 안에 둘

→ ORM도 같은 모양: `findOne` → 조건 확인 → `save`, 사이에 await 두 번

© 한상훈 NestJS 백엔드 마스터

시간축으로 본 두 요청

시각	요청 A (PC)	요청 B (폰)
0ms	find 시작, await	
0ms		find 시작, await
10ms	used: false 확인, markUsed 시작, await	
10ms		used: false 확인, markUsed 시작, await
20ms	할인 적용	
20ms		할인 적용

→ 스레드는 하나, 실행은 번갈아

© 한상훈 NestJS 백엔드 마스터

경쟁 상태

→ 경쟁 상태(race condition): 결과가 실행 순서와 끼어드는 시점에 따라 달라지는 상황

순서

A 다 끝나고 B: 할인 한 번 (정상)

번갈아

A와 B가 번갈아: 할인 두 번

문제

스레드 수와 무관, 끼어들 틈이 있는지가 문제

© 한상훈 NestJS 백엔드 마스터

임계 구역

```
const coupon = await db.find(code);          // ┐
if (coupon.used) return ...;                 // │ 임계 구역
await db.markUsed(code);                     // ┘
```

→ 임계 구역(critical section): 한 번에 한 요청만 지나가야 하는 구간

→ 이 구간 안의 await 두 번 = 틈 두 군데

© 한상훈 NestJS 백엔드 마스터

바로잡기 ①

"await 한 번마다 다른 요청이 끼어듭니다.
읽고, 기다리고, 쓰는 사이가 전부 틈입니다."

→ 잘못된 생각: Node는 싱글 스레드라 경쟁 상태 없음

© 한상훈 NestJS 백엔드 마스터

원자적 연산

→ 원자적 연산(atomic operation): 중간에 아무도 끼어들 수 없는 한 덩어리 연산

→ 밖에서 보면: 시작 전 아니면 끝난 뒤, 중간 상태는 안 보임

→ Node 메모리 안의 값: 동기 코드 한 덩어리는 끼어들 수 없음

→ 쓸 수 있는 곳: 이 프로세스 안에만 있는 값 (메모리 캐시, 요청 횟수 세기)

© 한상훈 NestJS 백엔드 마스터

확인과 기록 사이에 await 없음

nowait.mjs

```
const coupons = new Map([['WELCOME-5000', { used: false }]]);
async function useCoupon(code) {
  await new Promise(r => setTimeout(r, 10));   // 요청 검증 같은 앞 작업
  const c = coupons.get(code);
  if (c.used) return '이미 사용한 쿠폰';
  c.used = true;                                // 확인 바로 다음 줄에서 기록
  return '5,000원 할인 적용';
}
```

○ ○ ○ 터미널

```
$ node nowait.mjs
[ '5,000원 할인 적용', '이미 사용한 쿠폰' ]
```

© 한상훈 NestJS 백엔드 마스터

진짜 스레드 여러 개: 워커 4개

atomics.mjs

```
const counter = new Int32Array(workerData.shared);   // SharedArrayBuffer
for (let i = 0; i < 10_000_000; i++) {
  if (workerData.mode === 'Atomics.add') Atomics.add(counter, 0, 1);
  else counter[0]++;
}
```

→ SharedArrayBuffer: 워커끼리 복사 없이 같이 쓰는 메모리

© 한상훈 NestJS 백엔드 마스터

counter[0]++와 Atomics.add

방식	1회차	2회차	3회차
`counter[0]++`	18,341,078	12,882,442	21,134,216
`Atomics.add`	40,000,000	40,000,000	40,000,000

→ 기대값: 워커 4개 × 1,000만 = 40,000,000

→ `counter[0]++`: 읽기, 더하기, 쓰기 세 단계 → 다른 스레드가 사이에 끼어듦

데이터 블록: 2026-10-26 강사 PC 실측 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

뮤텍스

→ 뮤텍스(mutex, mutual exclusion): 한 번에 하나만 들어가게 하는 자물쇠

→ `lock`: 비어 있으면 잠그고 들어감, 잠겨 있으면 줄 서서 대기

→ `unlock`: 줄의 다음 사람에게 넘김

→ 쓰는 때: 임계 구역 안에 꼭 await가 있어야 할 때 (DB 읽기·쓰기)

→ NestJS: 서비스 필드에 하나 (서비스는 기본으로 앱에 하나), 요청마다 새로 만들면 무효

© 한상훈 NestJS 백엔드 마스터

프로미스로 만든 Mutex

sync.mjs

```
export class Mutex {
  #locked = false;
  #waiters = [];
  async lock() {
    if (!this.#locked) { this.#locked = true; return; }
    await new Promise(r => this.#waiters.push(r));   // 풀릴 때까지 대기 줄로
  }
  unlock() {
    const next = this.#waiters.shift();
    if (next) next(); else this.#locked = false;     // 다음 대기자에게 바로 넘김
  }
}
```

© 한상훈 NestJS 백엔드 마스터

뮤텍스를 건 쿠폰 사용

```
async function useCoupon() {
  await m.lock();
  try {
    await sleep(10);                    // DB에서 읽기
    if (used) return '이미 사용한 쿠폰';
    await sleep(10);                    // DB에 쓰기
    used = true;
    return '5,000원 할인 적용';
  } finally { m.unlock(); }
}
```

→ `finally`: 예외가 나도 반드시 unlock

○ ○ ○ 터미널

```
뮤텍스, 동시에 2번: [ '5,000원 할인 적용', '이미 사용한 쿠폰' ]
```

© 한상훈 NestJS 백엔드 마스터

뮤텍스 컨텐션

→ 뮤텍스 컨텐션(mutex contention): 같은 락을 기다리는 요청이 쌓이는 상태

→ 락을 넓게 걸수록: 서로 상관없는 요청도 한 줄

→ 증상: CPU는 한가한데 응답이 느림

→ 락 하나 + 임계 구역 20ms → 초당 50건이 상한

→ 임계 구역 안의 외부 호출: 결제사가 1초면 같은 락의 모든 요청이 1초씩 밀림

© 한상훈 NestJS 백엔드 마스터

락 하나와 쿠폰마다 락

락	서로 다른 쿠폰 50건 (임계 구역 20ms)
서버 전체에 락 하나	1,012ms
쿠폰 코드마다 락	21ms

```
const perKey = new Map();
const lockFor = (k) => perKey.get(k) ?? perKey.set(k, new Mutex()).get(k);
```

→ 락은 지킬 데이터 단위로

데이터 블록: 2026-10-26 WSL2, Node v22.22.0 실측 (부록 시트), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

세마포어

→ 세마포어(semaphore): 한 번에 N개까지 들어가게 하는 자물쇠

→ N = 1이면 뮤텍스와 같음

→ `acquire`: 자리가 있으면 하나 쓰고 들어감, 없으면 대기

→ `release`: 자리 반납, 줄의 다음 사람 입장

→ 예: 결제사의 동시 요청 한도 5개 (가정)

© 한상훈 NestJS 백엔드 마스터

결제사 동시 호출 5개

```
const s = new Semaphore(5);
async function callPg() {
  await s.acquire();
  try { await sleep(100); } finally { s.release(); }   // 결제사 호출 100ms
}
await Promise.all(Array.from({ length: 50 }, callPg));
```

○ ○ ○ 터미널

```
결제 50건, 세마포어 5: 최대 동시 호출 5, 전체 1003ms
```

→ 넘는 요청: 거절 대신 우리 쪽에서 대기

→ 대기에도 한도: 예) 2초 넘게 자리가 없으면 "결제 요청이 많음" 즉시 응답

© 한상훈 NestJS 백엔드 마스터

조건 변수

→ 조건 변수(condition variable): 어떤 조건이 참이 될 때까지 잠들어 기다리고, 조건을 바꾼 쪽이 깨워 주는 도구

→ 워커 풀: "빈 워커가 있다"가 참이 될 때까지 작업이 줄에서 대기

→ 워커가 끝나면: 줄의 다음 작업을 깨움

→ Node에서: 대기 줄에 넣은 resolve + 조건을 바꾼 쪽에서 호출

→ 계속 확인하는 반복문(바쁜 대기)은 루프를 멈춤

→ 기다릴 때는 언제나 await

© 한상훈 NestJS 백엔드 마스터

네 가지 도구

도구	한 번에 들어가는 수	예
원자적 연산	끼어들 틈 자체를 없앰	await 없는 확인·기록, `Atomics.add`
뮤텍스	1	같은 쿠폰 사용
세마포어	N	결제사 동시 호출 5개
조건 변수	조건이 참이 될 때	빈 워커를 기다리는 작업

© 한상훈 NestJS 백엔드 마스터

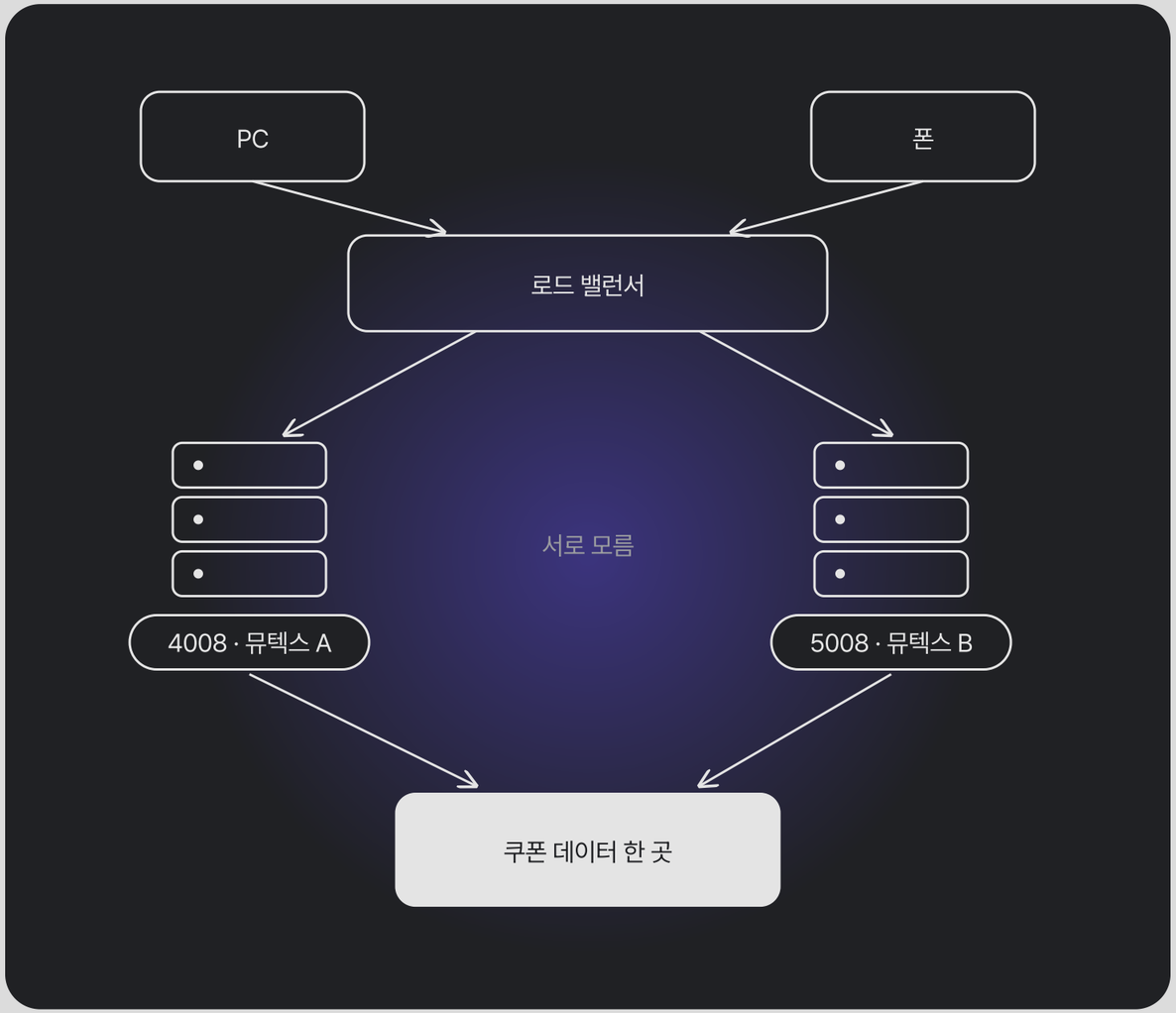
서버 두 대, 뮤텍스 두 개

→ PC → 로드 밸런서 → 4008번 서버 (뮤텍스 A)

→ 폰 → 로드 밸런서 → 5008번 서버 (뮤텍스 B)

→ 쿠폰 데이터: 한 곳

→ 뮤텍스 A와 B는 서로 모름



© 한상훈 NestJS 백엔드 마스터

각자 뮤텍스를 건 두 서버

터미널

```
$ node two-fire.mjs
--- /coupons/use: PC는 4008번 서버, 폰은 5008번 서버
[4008] 5,000원 할인 적용
[5008] 5,000원 할인 적용
```

→ 프로세스마다 뮤텍스를 걸었는데도 할인 두 번

→ 두 번 실행, 두 번 모두 같은 결과

© 한상훈 NestJS 백엔드 마스터

데이터가 있는 곳에서 한 번에

```
// 데이터를 가진 쪽: 확인과 기록을 한 덩어리로
if (url.pathname === '/db/use-if-unused') {
  if (c.used) return res.end('{"changed":0}');
  c.used = true; return res.end('{"changed":1}');
}
```

○ ○ ○ 터미널

```
--- /coupons/use-atomic: PC는 4008번 서버, 폰은 5008번 서버
[4008] 5,000원 할인 적용
[5008] 이미 사용한 쿠폰
```

→ SQL이라면: `UPDATE coupons SET used = true WHERE code = $1 AND used = false` → 바뀐 행 수 확인

© 한상훈 NestJS 백엔드 마스터

틈을 찾는 법

→ 같은 데이터를 읽고, await, 그 값을 바탕으로 쓰는 코드

→ 쿠폰 사용, 남은 좌석 차감, 포인트 적립, 재고 차감, 1인당 구매 한도

→ 막는 순서: await 없앨 수 있나 → 데이터가 있는 곳에서 한 번에 할 수 있나 → 프로세스 안 락은 서버 한 대일 때만

→ 찾은 곳: 동시 요청이 부딪히는 곳 목록에 한 줄씩

→ 테스트: 같은 쿠폰으로 요청 10개를 `Promise.all`로, 성공이 정확히 1번인지 확인

© 한상훈 NestJS 백엔드 마스터

await 사이의 틈

"await 한 번마다 다른 요청이 끼어듭니다.
읽고, 기다리고, 쓰는 사이가 전부 틈입니다."

→ 프로세스 안의 락은 그 프로세스만 막음